

Full Stack Assignment: Smart Parking Management System

Note to Candidates:

You are free to use any AI-based tools or editors (e.g., GitHub Copilot, ChatGPT, etc.) while completing this assignment. However, the goal of this challenge is to assess your **understanding of the code**, your ability to **explain technical decisions**, and your skill in **debugging or modifying existing implementations**. Please be prepared to walk through your code, justify your design, and address potential improvements or bugs during the technical interview.

Tech Stack:

- **Frontend:** Angular
 - **Backend:** Python (FastAPI preferred), SQLAlchemy, PostgreSQL
 - **Caching:** Redis (or in-memory fallback if Redis is unavailable)
 - **Queueing:** Celery + Redis (or Python `queue` as a fallback)
-

Scenario Overview:

You are building a **Smart Parking Management System** that supports both cars and bikes, dynamic pricing, and real-time availability. Users should be able to book parking slots based on vehicle type, pricing rules, and available time windows. Admins should manage lots and generate reports. Empty car slots can accommodate two bikes.

Functional Requirements:

User Features:

- **Search Availability:**
 - Choose vehicle type: `bike` or `car`
 - Select time range and parking lot
 - View available slots and dynamic pricing
- **Booking:**
 - Fill form with name, vehicle number, vehicle type
 - Prevent double bookings
 - Confirm with booking ID
- **Cancel Booking:**
 - Use booking ID to cancel
 - Release slot and update availability instantly
- **Pricing Rules:**

- Dynamic pricing by time and lot (e.g., peak/off-peak hours)
 - Example: Lot A - \$5/hr (6PM–10PM), \$2/hr (rest)
 - **Space Logic:**
 - 1 car slot can accommodate 2 bikes when empty
 - Bike booking logic must consider splitting and reusing car slots
-

Admin Features:

- Add/Remove parking lots
 - Add/Remove slots (type: car/bike)
 - Configure time-based pricing rules
 - View revenue reports by day or lot
-

Backend Requirements:

- REST API (FastAPI) with OpenAPI schema
 - Models:
 - ParkingLot, ParkingSlot, Booking, User, PricingRule
 - Slot Allocation Logic:
 - Support car/bike dynamics
 - Handle partial car slot usage for bikes
 - Prevent overbooking
 - Pricing Computation:
 - Apply time-based pricing
 - Caching:
 - Cache real-time availability (Redis preferred)
 - Invalidate cache on booking or cancel
 - Queueing:
 - Use Celery or similar for background tasks (e.g., report generation, notifications)
-

Algorithmic Challenge:

Write efficient logic to allocate slots based on:

- N car slots
- M bike slots
- K bikes
- L cars

Optimize for:

- Space utilization
 - Booking order fairness
-

Frontend Requirements:

- Form to select lot, time, vehicle type
 - Availability display with pricing
 - Booking confirmation view
 - Cancel booking screen
 - Responsive UI with Angular best practices
 - Angular service layer for API abstraction
 - Bonus: WebSocket/polling for real-time updates
-

Bonus Features:

- JWT Authentication for user sessions
 - Booking expiry logic (e.g., auto-expire after 10 mins)
 - Export revenue reports to CSV
 - Occupancy heatmaps or charts
 - Rate limiting and debouncing for API calls
-

Deliverables:

- GitHub Repo or ZIP file containing:
 - `/frontend` Angular app
 - `/backend` FastAPI app
 - README with clear setup steps
 - Seed script or DB dump with sample data
 - Postman collection (optional)